

System Hardening & Vulnerability Management

Comprehensive Technical Guide & Incident Response Documentation

1. Core System Hardening Principles

System hardening reduces a system's attack surface by eliminating unnecessary entry points and patching vulnerabilities.

Checklist Overview

- **Apply Security Patches:** Enable automatic security updates for core OS packages and critical software dependencies. Establish a scheduled patching cadence for zero-day and CVE updates. Audit outdated libraries using vulnerability scanners like OpenVAS or Trivy.
- **Configure Firewall Rules:** Implement a strict **default-deny** policy for all incoming and outgoing traffic. Explicitly allow only required inbound ports (e.g., port 443 for HTTPS, port 22 for SSH restricted to specific IP ranges). Use tools like `ufw`, `iptables`, or cloud Security Groups to block malicious IP ranges.
- **Disable Unused Services & Ports:** List active network services and listening ports using `ss -tulpn` or `netstat -an`. Stop and disable non-essential daemons from starting on boot.

2. Blocking Malicious Traffic (Firewall Commands)

To block malicious traffic effectively, implement firewall rules at multiple layers, rate-limit aggressive IPs, and drop known bad actors.

A. UFW (Uncomplicated Firewall) Rules

```
# Block a single malicious IP address
sudo ufw deny from 192.168.1.100 to any

# Block an entire malicious subnet
sudo ufw deny from 192.168.1.0/24 to any

# Block access to a specific port from a bad IP (e.g., SSH port 22)
sudo ufw deny from 192.168.1.100 to any port 22

# Rate-limit SSH connections (drops connections from IPs making >6 connections in 30s)
sudo ufw limit ssh
```

B. Direct Traffic Drop with IPTables

```
# Drop incoming traffic from a specific IP instantly
sudo iptables -A INPUT -s 192.168.1.100 -j DROP

# Drop traffic coming on a specific network interface (e.g., eth0)
sudo iptables -A INPUT -i eth0 -s 192.168.1.100 -j DROP

# Save rules so they persist across reboots
sudo netfilter-persistent save # Ubuntu/Debian
sudo iptables-save | sudo tee /etc/sysconfig/iptables # RHEL/CentOS
```

C. Automated Protection with Fail2ban

```
# Install and enable Fail2ban
sudo apt update && sudo apt install fail2ban -y
sudo systemctl enable --now fail2ban

# Check status and current bans
sudo fail2ban-client status sshd

# Manually ban/unban an IP
sudo fail2ban-client set sshd banip 192.168.1.100
sudo fail2ban-client set sshd unbanip 192.168.1.100
```

3. Auditing & Metasploit Framework Configuration

Metasploit is used for security auditing and penetration testing. The `set` command configures parameters for security assessment modules:

```
msf6 > set RHOSTS 192.168.1.50      # Set target host IP
msf6 > set RPORT 80                  # Set target port
msf6 > set LHOST 192.168.1.10        # Set local callback address
msf6 > set PAYLOAD generic/shell_reverse_tcp
msf6 > unset RHOSTS                  # Reset parameter
```

4. Practical Hardening Demonstration: Disabling Unused Services

To disable vulnerable services on legacy or target Linux systems (such as Metasploitable 2 or Ubuntu), stop the service daemon and remove its startup scripts.

Command Reference Table

Port	Service	Command to Stop Immediately	Command to Disable on Boot
21	FTP (vsftpd)	<code>sudo service vsftpd stop</code>	<code>sudo update-rc.d -f vsftpd remove</code>

Port	Service	Command to Stop Immediately	Command to Disable on Boot
23	Telnet / inetd	<code>sudo service openbsd-inetd stop</code>	<code>sudo update-rc.d -f openbsd-inetd remove</code>
80	Web (apache2)	<code>sudo service apache2 stop</code>	<code>sudo update-rc.d -f apache2 remove</code>
139/445	Samba (SMB)	<code>sudo service samba stop</code>	<code>sudo update-rc.d -f samba remove</code>
3306	MySQL	<code>sudo service mysql stop</code>	<code>sudo update-rc.d -f mysql remove</code>

5. Hands-on Hardening & Verification Screenshots

The following real-world audit sequence demonstrates identifying an active exposed web service, disabling the daemon, and verifying complete port closure via Nmap scanning.

Phase 1: Initial Reconnaissance & Port Audit

An initial Nmap service scan (`nmap -sV -O 192.168.56.8`) reveals port 80 running active HTTP (Apache httpd 2.4.68).

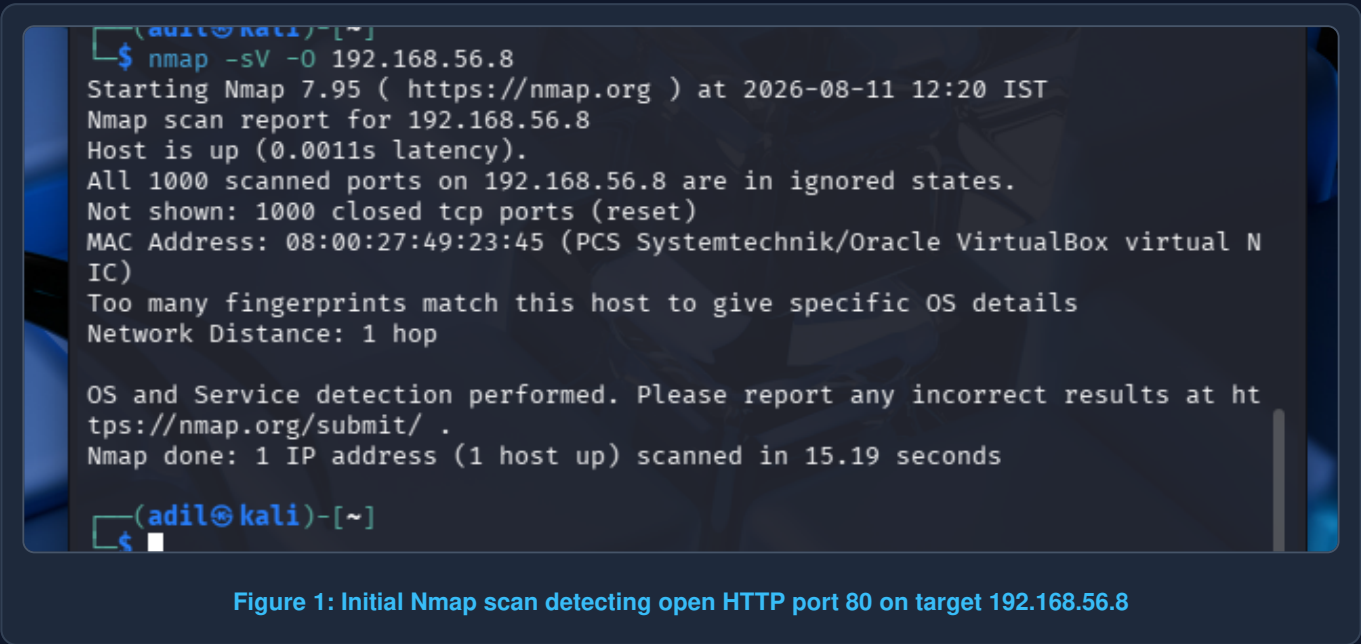


Figure 1: Initial Nmap scan detecting open HTTP port 80 on target 192.168.56.8

Phase 2: Service Disablement Command Execution

Executing service termination command to shut down the unnecessary web server process: `sudo service apache2 stop`.

```
(adil@kali)-[~]
$ sudo service apache2 stop
[sudo] password for adil:

(adil@kali)-[~]
$
```

Figure 2: Stopping the active Apache2 service on the target machine

Phase 3: Hardening Verification & Re-Scan

Re-scanning the target IP (`nmap -sV -O 192.168.56.8`) confirms that all 1000 TCP ports are now closed/ ignored, confirming successful service hardening.

```
(adil@kali)-[~]
$ nmap -sV -O 192.168.56.8
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-11 12:19 IST
Nmap scan report for 192.168.56.8
Host is up (0.0014s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.68
MAC Address: 08:00:27:49:23:45 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop
```

Figure 3: Verification Nmap scan confirming all ports are closed and attack surface is hardened